

DATABASE 2026

IZBAI NURADIL

LAB 4

1. Retrieve all airline names in uppercase.

The screenshot shows a PostgreSQL query editor interface. The query editor is titled "postgres/postgres@PostgreSQL 18" and contains the following SQL query:

```
1 SELECT UPPER(airline_name) AS airline_name
2 FROM airline;
```

The query is executed, and the results are displayed in the "Data Output" tab. The results show 9 rows of data, with the column "airline_name" of type "text". The rows are:

airline_name
AIR ASTANA
QAZAQ AIR
LUFTHANSA
TURKISH AIRLIN...
KAZAIR
AIREASY
FLYHIGH
FLYFLY
GLOBAL AIRWAYS

The status bar at the bottom indicates "Total rows: 9" and "Query complete 00:00:00.074".

2. Replace any occurrence of the word "Air" in airline names with "Aero".

The screenshot shows a PostgreSQL client interface with a query editor and a data output window. The query editor contains the following SQL code:

```
1 SELECT REPLACE(airline_name, 'Air', 'Aero') AS modified_name
2 FROM airline;
```

The data output window displays the results of the query, showing a table with one column, `modified_name`, and nine rows of data:

modified_name
Aero Astana
Qazaq Aero
Lufthansa
Turkish Aerolines
KazAero
AeroEasy
FlyHigh
FlyFly
Global Aeroways

The status bar at the bottom indicates "Total rows: 9" and "Query complete 00:00:00.056".

3. Find all flight numbers that coordinates with both airline 1 and airline 2.

The screenshot shows a PostgreSQL client interface with a query editor and a data output window. The query editor contains the following SQL code:

```
1 SELECT flight_id
2 FROM flights
3 WHERE airline_id = 1
4 INTERSECT
5 SELECT flight_id
6 FROM flights
7 WHERE airline_id = 2;
```

The data output window displays the results of the query, showing a table with one column, `flight_id`, and no rows of data.

flight_id

The status bar at the bottom indicates "Total rows: 0" and "Query complete 00:00:00.047".

4.Retrieve airports that contain the word "Reginal" and "Air" in their names.

The screenshot shows the PostgreSQL query editor interface. The query is as follows:

```
1 SELECT *
2 FROM airports
3 WHERE airport_name LIKE '%Reginal%'
4 AND airport_name LIKE '%Air%';
```

The Data Output tab is active, showing the schema of the 'airports' table:

airport_code	airport_name	city	country	latitude	longitude	timezone
[PK] character varying (10)	character varying (50)	character varying (50)	character varying (50)	numeric (9,6)	numeric (9,6)	character varying (10)

Total rows: 0 | Query complete 00:00:00.060 | LF | Ln 4, Col 33

5.Retrieve passenger names and format their birth dates as 'Month DD, YYYY'.

The screenshot shows the PostgreSQL query editor interface. The query is as follows:

```
1 SELECT first_name, last_name, TO_CHAR(date_of_birth, 'Month DD, YYYY') AS formatted_birth_date
2 FROM passengers;
```

The Data Output tab is active, showing the results of the query:

first_name	last_name	formatted_birth_date
Ali	Zhan	May 12, 1995
Algerim	Kassymova	September 20, 1998
John	Doe	March 15, 1985
Aylin	Yilmaz	November 05, 1992
Omar	Al-Maktoum	July 07, 1990

Showing rows: 1 to 5 | Page No: 1 of 1 | LF | Ln 2, Col 17

6. Find flight numbers that have been delayed based on the actual arrival time.

The screenshot shows a PostgreSQL client interface with a query editor and a results pane. The query editor contains the following SQL query:

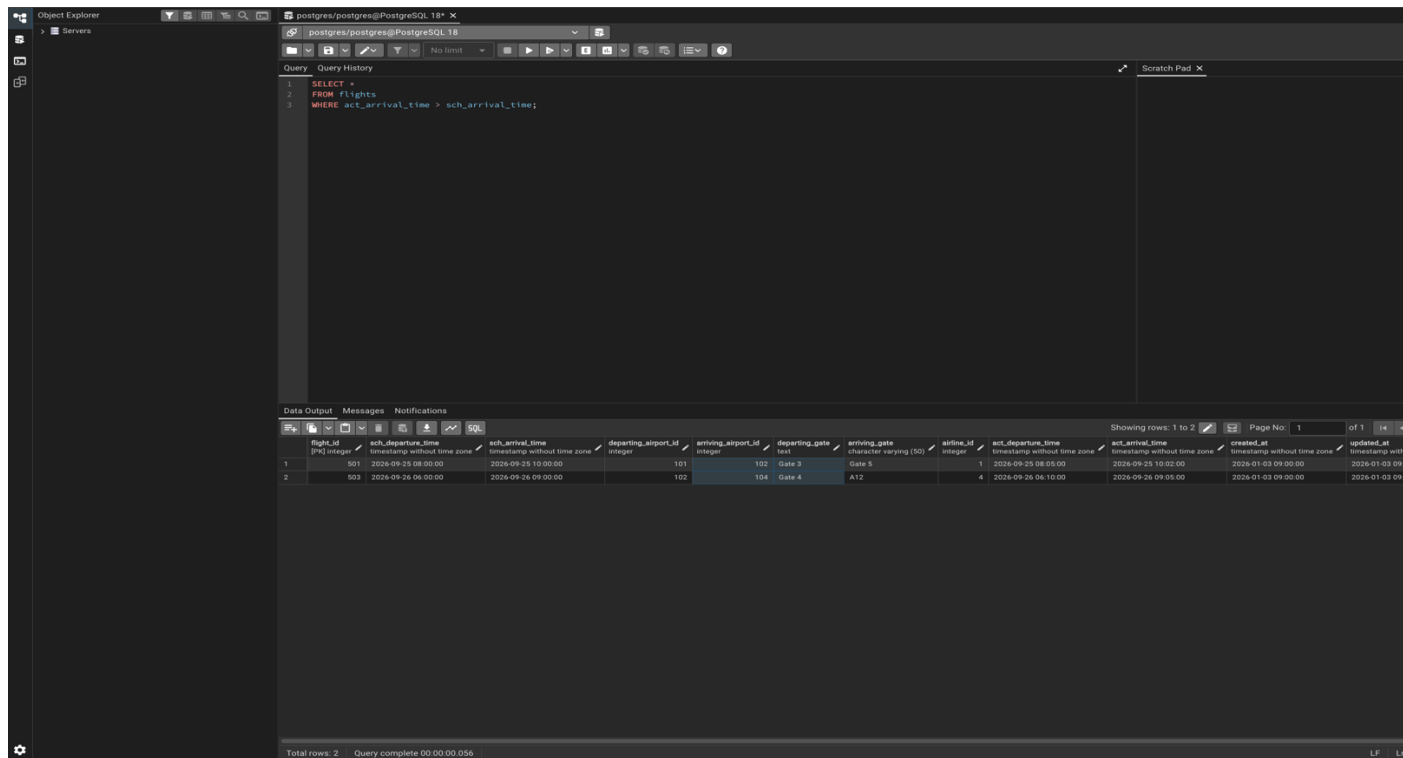
```
1 SELECT *
2 FROM flights
3 WHERE act_arrival_time > sch_arrival_time;
```

The results pane displays the following data:

flight_id [PK] integer	sch_departure_time timestamp without time zone	sch_arrival_time timestamp without time zone	departing_airport_id integer	arriving_airport_id integer	departing_gate text	arriving_gate character varying (50)	airline_id integer	act_dep timestamp	
1	501	2026-09-25 08:00:00	2026-09-25 10:00:00	101	102	Gate 3	Gate 5	1	2026-0
2	503	2026-09-26 06:00:00	2026-09-26 09:00:00	102	104	Gate 4	A12	4	2026-0

The interface also shows a status bar at the bottom indicating "Total rows: 2" and "Query complete 00:00:00.056".

7. Find flights that arrived late according to their actual arrival time compared to the scheduled arrival time.

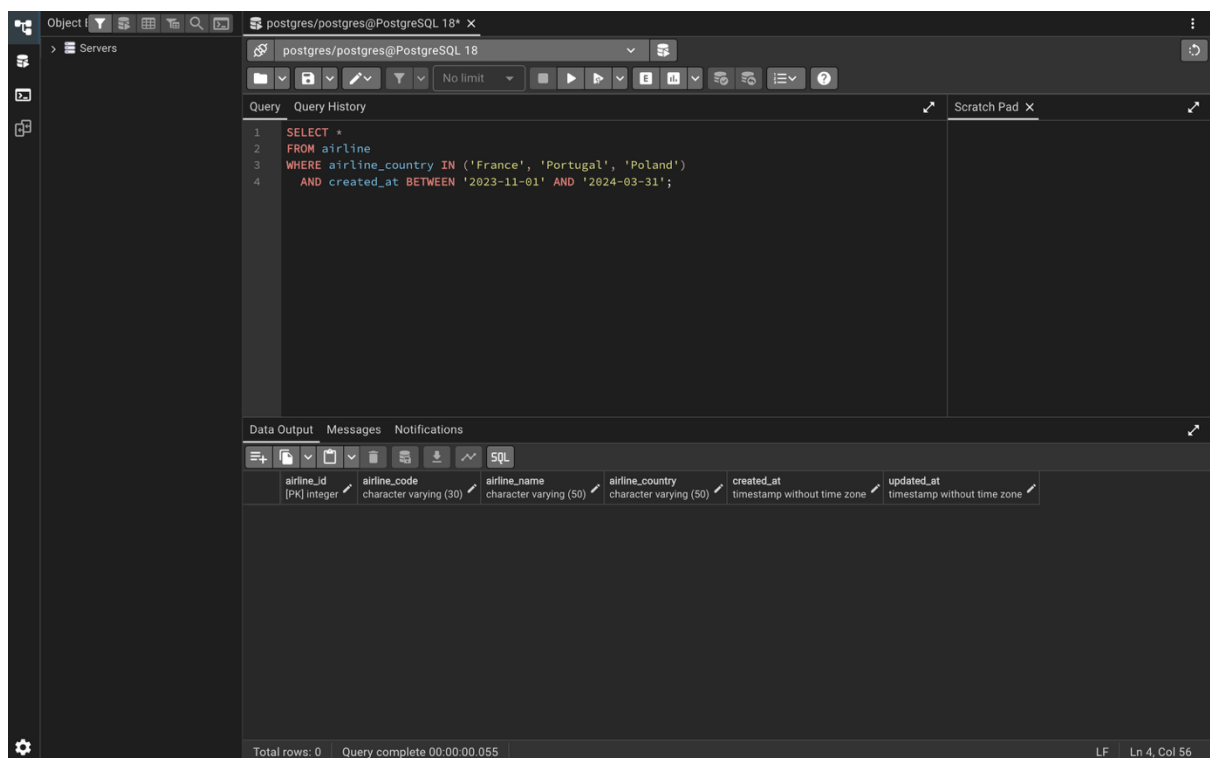


```
1 SELECT *
2 FROM flights
3 WHERE act_arrival_time > sch_arrival_time;
```

	flight_id [PK] integer	sch_departure_time timestamp without time zone	sch_arrival_time timestamp without time zone	departing_airport_id integer	arriving_airport_id integer	departing_gate text	arriving_gate character varying (50)	airline_id integer	act_departure_time timestamp without time zone	act_arrival_time timestamp without time zone	created_at timestamp without time zone	updated_at timestamp without time zone
1	501	2026-09-25 08:00:00	2026-09-25 10:00:00	101	102	Gate 3	Gate 5	1	2026-09-25 08:05:00	2026-09-25 10:02:00	2026-01-03 09:00:00	2026-01-03 09:00:00
2	503	2026-09-26 06:00:00	2026-09-26 09:00:00	102	104	Gate 4	A12	4	2026-09-26 06:10:00	2026-09-26 09:05:00	2026-01-03 09:00:00	2026-01-03 09:00:00

Total rows: 2 | Query complete 00:00:00.056

8. Show airlines from any of: ('France','Portugal','Poland') created between '2023-11-01' and '2024-03-31'.



```
1 SELECT *
2 FROM airline
3 WHERE airline_country IN ('France', 'Portugal', 'Poland')
4 AND created_at BETWEEN '2023-11-01' AND '2024-03-31';
```

airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
-------------------------	-------------------------------------	-------------------------------------	--	--	--

Total rows: 0 | Query complete 00:00:00.055

9. Find top3 overweighted baggage with more than 25kg.

The screenshot shows a PostgreSQL query editor interface. The query is as follows:

```
1 SELECT *
2 FROM baggage
3 WHERE weight_in_kg > 25
4 ORDER BY weight_in_kg DESC
5 LIMIT 3;
```

The query has been executed, and the results are displayed in the Data Output pane. The results show one row of data:

baggage_id [PK] integer	weight_in_kg numeric (4,2)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	
1	14	30.50	2026-09-26 12:10:00	2026-09-26 12:10:00	9004

The status bar at the bottom indicates "Total rows: 1" and "Query complete 00:00:00.058".

10. Find the youngest passengers' full name.

The screenshot shows a PostgreSQL query editor interface. The query is as follows:

```
1 SELECT a.airline_name, COUNT(f.flight_id) AS total_flights
2 FROM airline a
3 LEFT JOIN flights f ON a.airline_id = f.airline_id
4 GROUP BY a.airline_name
5 ORDER BY total_flights DESC;
```

The query has been executed, and the results are displayed in the Data Output pane. The results show 9 rows of data:

airline_name character varying (50)	total_flights bigint
1 Air Astana	2
2 Lufthansa	1
3 Turkish Airlines	1
4 Global Airways	1
5 KazAir	0
6 Qazaq Air	0
7 FlyHigh	0
8 AirEasy	0
9 FlyFly	0

The status bar at the bottom indicates "Total rows: 9" and "Query complete 00:00:00.055".

11. Find the cheapest booking price on each booking platform.

The screenshot shows a PostgreSQL query editor with the following query:

```
1 SELECT DISTINCT a.*
2 FROM airports a
3 JOIN flights f ON a.airport_code = f.arriving_airport_id::text;
```

The Data Output panel shows the following columns:

airport_code	airport_name	city	country	latitude	longitude	timezone
[PK] character varying (10)	character varying (50)	character varying (50)	character varying (50)	numeric (9,6)	numeric (9,6)	character varying (10)

Total rows: 0 | Query complete 00:00:00.054 | LF | Ln 3, Col 64

12. Return airlines whose airline_code contains a digit.

The screenshot shows a PostgreSQL query editor with the following query:

```
1 SELECT DISTINCT p.first_name, p.last_name, p.date_of_birth
2 FROM passengers p
3 JOIN booking b ON p.passenger_id = b.passenger_id
4 JOIN booking_flight bf ON b.booking_id = bf.booking_id
5 JOIN flights f ON bf.flight_id = f.flight_id
6 JOIN airline a ON f.airline_id = a.airline_id;
```

The Data Output panel shows the following columns:

first_name	last_name	date_of_birth	
character varying (50)	character varying (50)	date	
1	Algerim	Kassymova	1998-09-20
2	Ali	Zhan	1995-05-12
3	Aylin	Yilmaz	1992-11-05
4	John	Doe	1985-03-15
5	Omar	Al-Maktoum	1990-07-07

Showing rows: 1 to 5 | Page No: 1 of 1 | LF | Ln 6, Col 47

13.List

The screenshot shows a PostgreSQL query editor with the following SQL query:

```
1 SELECT DISTINCT p.first_name, p.last_name, p.date_of_birth
2 FROM passengers p
3 JOIN booking b ON p.passenger_id = b.passenger_id
4 JOIN booking_flight bf ON b.booking_id = bf.booking_id
5 JOIN flights f ON bf.flight_id = f.flight_id
6 JOIN airline a ON f.airline_id = a.airline_id;
```

The Data Output tab shows the results of the query:

	first_name character varying (50)	last_name character varying (50)	date_of_birth date
1	Algerim	Kassymova	1998-09-20
2	Ali	Zhan	1995-05-12
3	Aylin	Yilmaz	1992-11-05
4	John	Doe	1985-03-15
5	Omar	Al-Maktoum	1990-07-07

Total rows: 5 Query complete 00:00:00.047

the top 5 most recently created airlines.

14.Return all rows where booking_id is between 200 and 300 inclusive and check_result <> 'Checked'.

The screenshot shows a PostgreSQL query editor with the following SQL query:

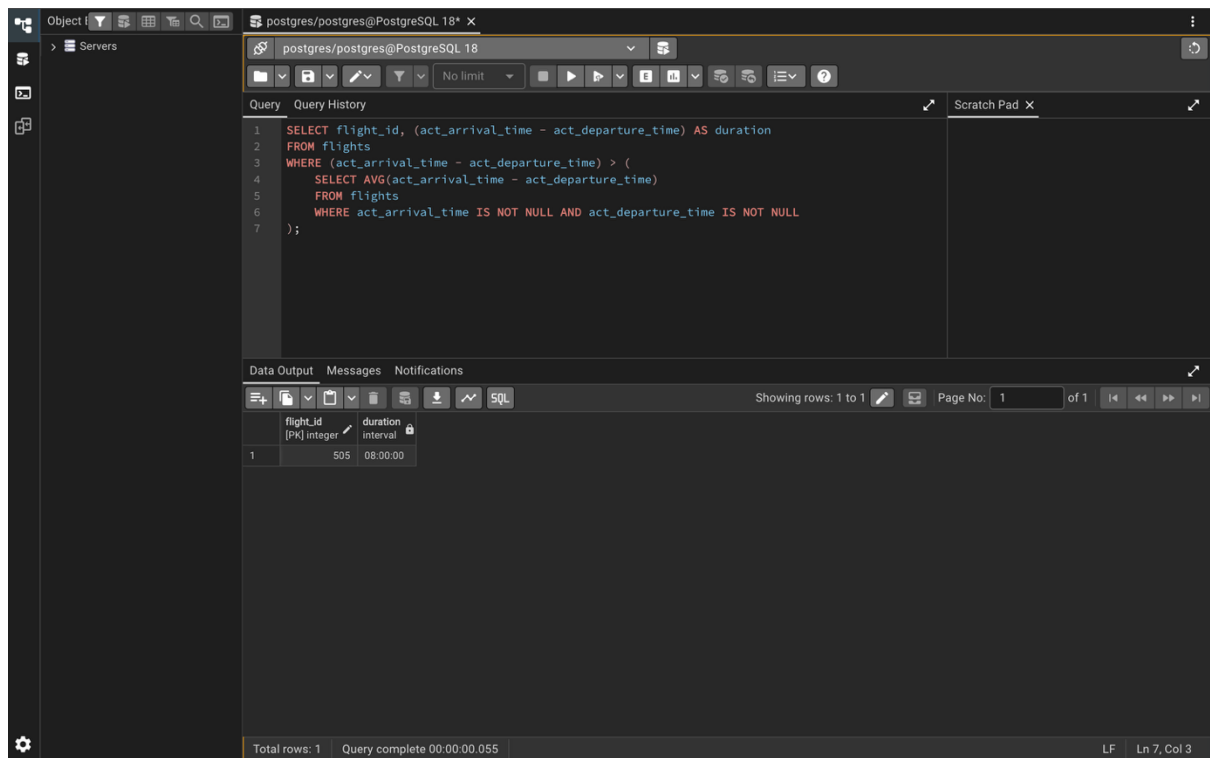
```
1 SELECT a.airport_code, a.airport_name, COUNT(f.flight_id) AS total_flights
2 FROM airports a
3 LEFT JOIN flights f ON a.airport_code = f.departing_airport_id::text
4 OR a.airport_code = f.arriving_airport_id::text
5 GROUP BY a.airport_code, a.airport_name
6 ORDER BY total_flights DESC;
```

The Data Output tab shows the results of the query:

	airport_code [PK] character varying (10)	airport_name character varying (50)	total_flights bigint
1	A01	Airport 1	0
2	A02	Airport 2	0
3	A03	Airport 3	0
4	A04	Airport 4	0
5	A05	Airport 5	0
6	A06	Airport 6	0
7	A07	Airport 7	0
8	A08	Airport 8	0
9	A09	Airport 9	0
10	A10	Airport 10	0
11	A11	Airport 11	0
12	A12	Airport 12	0
13	A13	Airport 13	0
14	A14	Airport 14	0
15	A15	Airport 15	0
16	A16	Airport 16	0

Total rows: 200 Query complete 00:00:00.060

15. Baggage checks where `update_at` is in the same month as `created_at` but occurs earlier than `created_at`.



The screenshot shows a PostgreSQL IDE interface. The top toolbar includes icons for file operations, search, and execution. The main editor displays a SQL query:

```
1 SELECT flight_id, (act_arrival_time - act_departure_time) AS duration
2 FROM flights
3 WHERE (act_arrival_time - act_departure_time) > (
4     SELECT AVG(act_arrival_time - act_departure_time)
5     FROM flights
6     WHERE act_arrival_time IS NOT NULL AND act_departure_time IS NOT NULL
7 );
```

Below the query editor, the 'Data Output' tab is active, showing a single row of results:

flight_id	duration
[PK] integer	interval
1	505 08:00:00

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.055'.